

Assignment #4:

Performance Analysis of GPU Kernels in GEM5

Žane Bučan, Stjepan Sazonov
RS 2025/26, FRI, UL

I. INTRODUCTION

This report analyzes the performance of GPU kernels simulated in GEM5. The assignment evaluates three workloads: histogram computation, matrix transpose, and sparse matrix-vector multiplication (SpMV). The goal is to compare naive and optimized implementations and to observe how compute-unit count, memory behavior, shared memory usage, and control-flow divergence affect performance.

II. TASK 1: HISTOGRAM KERNEL

A. Naive and Privatized Implementations

The first workload computes a 256-bin histogram of an input image. The naive implementation updates one global histogram, which creates heavy contention. The optimized implementation privatizes the histogram per group of threads and later merges the private histograms into one global result.

TABLE I
HISTOGRAM PERFORMANCE

CU	Kernel	Latency	vALU	LDS	Cycles	VPC
2	Naive	6150268.68	22528	0	827250	3.8026
2	Priv.	806751.71	28672	393216	229998	21.6556
4	Naive	12392765.14	11088	0	805317	1.9226
4	Priv.	1673886.96	14336	196608	176513	14.1087
8	Naive	24162123.17	5632	0	784254	1.0028
8	Priv.	4135812.01	7168	98304	171581	7.2571

The optimized version is significantly faster for all compute-unit configurations. At two compute units, total cycles decrease from 827250 to 229998, and VPC increases from 3.8026 to 21.6556. This shows that privatization reduces global-memory contention and allows much higher vector throughput.

As the number of compute units increases, the naive kernel does not scale well. Its mean load latency rises from about 6.15 million to 24.16 million, while VPC drops from 3.8026 to 1.0028. The privatized version also shows higher latency with more compute units, but it remains much faster because most histogram updates are moved to LDS. The LDS access count decreases with more compute units because the average work per compute unit becomes smaller.

III. TASK 2: MATRIX TRANSPOSE

A. Naive and Shared-Memory Implementations

The second workload transposes a 512×512 matrix. The naive implementation directly reads and writes global memory,

while the optimized version uses a 16×17 LDS tile. The extra column reduces bank conflicts during tiled transposition.

TABLE II
MATRIX TRANSPOSE PERFORMANCE

CU	Kernel	Latency	vALU	LDS	Cycles	VPC
2	Naive	8028555.91	36360	0	526860	7.3613
2	Shared	2676883.30	53560	263680	326839	21.3791
4	Naive	19325775.39	18288	0	590495	3.3035
4	Shared	6201466.80	25896	127488	331883	10.1796
8	Naive	42430520.02	9216	0	666437	1.4751
8	Shared	12942396.97	13312	65536	331165	5.2442

The shared-memory implementation outperforms the naive version for every compute-unit count. At two compute units, cycles decrease from 526860 to 326839 and VPC increases from 7.3613 to 21.3791.

The naive transpose becomes worse as the number of compute units increases: mean load latency grows from 8.03 million to 42.43 million, cycles increase, and VPC drops. The shared memory kernel also experiences higher latency with more compute units.

IV. TASK 3: SPMV AND THREAD DIVERGENCE

A. Divergent and Uniform Row Ordering

The third workload evaluates a CSR SpMV kernel. In the divergent version, rows are processed in their original order, where row lengths vary from 1 to 64 nonzeros. In the uniform version, rows are sorted by nonzero count before execution, grouping rows with similar work together.

TABLE III
SPMV DIVERGENCE AND MEMORY BEHAVIOR

CU	Kernel	Div. mean	Div. stdev	vALU	Reads	Coalesced
2	Div.	33.496	19.052	6328	1544	3104
2	Uniform	61.015	10.370	3256	776	1664
4	Div.	33.496	19.053	3164	772	3104
4	Uniform	60.620	10.974	1436	340	1664
8	Div.	33.496	19.055	1582	386	3104
8	Uniform	59.408	12.577	526	122	1664

The sorted implementation performs less total work in the measured statistics. For example, at eight compute units, vALU instructions decrease from 1582 to 526 and global reads decrease from 386 to 122. The number of coalesced line addresses also decreases from 3104 to 1664 for all compute-unit counts. This indicates that sorting rows by length improves the regularity of execution and memory behavior.

The divergence mean is higher for the uniform version, but its standard deviation is much lower. This means that execution becomes more consistent across different runs even though the reported mean value increases. In contrast, the divergent implementation has a standard deviation around 19 for all configurations, reflecting the non-uniform row lengths in the original CSR ordering.

V. CONCLUSION

The experiments show that GPU kernel performance in GEM5 is strongly affected by memory behavior and workload regularity. For the histogram kernel, privatization greatly reduces global-memory contention and improves throughput, although it introduces many LDS accesses. The naive histogram kernel scales poorly because more compute units increase contention on the shared global histogram.

For matrix transpose, the shared-memory tiled implementation consistently outperforms the naive version. The 16×17 LDS tile improves locality and avoids the worst effects of non-coalesced global-memory writes. Even though the shared-memory kernel performs more vector ALU work, the reduction in memory latency and cycles makes it faster.

For SpMV, sorting rows by nonzero count improves execution regularity and reduces the number of vector ALU instructions, global reads, and coalesced line accesses. Overall, the results demonstrate that optimization on GPUs depends not only on increasing parallelism, but also on reducing contention, using shared memory effectively, and limiting divergence between threads in the same wavefront.